```
import numpy as np,pandas as pd,matplotlib.pyplot as plt
from sklearn.datasets import make_blobs
```

```
x,y_true = make_blobs(n_samples=300, centers=3,cluster_std=1.2,random_state=42)
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
```

```
X_train,X_test,y_train,y_test = train_test_split(x,y_true,test_size=0.2,random_state=42)
clf = LogisticRegression()
clf.fit(X_train,y_train)
print(f'supervised accuracy:{clf.score(X_test,y_test):.2%}')
```

```
supervised accuracy:100.00%
```

```
from sklearn.cluster import KMeans
KMeans = KMeans(n_clusters=3,random_state=42,n_init=10)
cluster_labels = KMeans.fit_predict(x)
print('Cluster assignments (first 10):',cluster_labels[:10])
```

```
Cluster assignments (first 10): [1 1 0 2 1 2 0 2 0 0]
```

```
x, _ = make_blobs(n_samples=200, centers=3, random_state=42)

centers = x[np.random.choice(x.shape[0], 3, replace=False)]

def manual_kmeans(x, k=3, iterations=10):

    global centers

    for i in range(iterations):

        dists = np.linalg.norm(x[:, np.newaxis] - centers, axis=2)

        labels = np.argmin(dists, axis=1)

        new_centers = np.array([x[labels == k].mean(axis=0) for k in range(3)])

        print(f'iter {i+1}: center moved by {np.linalg.norm(new_centers - centers)}')

        if np.allclose(centers, new_centers):
            print("=> converged")
            break

        centers = new_centers

    return labels, centers


labels, centers = manual_kmeans(x, k=3)
```

```
iter 1: center moved by 7.316537486220843
iter 2: center moved by 6.621829934285335
iter 3: center moved by 0.0
=> converged
```

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import seaborn as sns
```

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import seaborn as sns
print('Loading MNIST dataset...')
(X_train_full, y_train_full), (X_test_full, y_test_full) = tf.keras.datasets.mnist.load_data()
X = np.concatenate((X_train_full, X_test_full), axis=0)
y = np.concatenate((y_train_full, y_test_full), axis=0)
X = X.reshape(X.shape[0], -1)
print(f'Full dataset: {X.shape}')
y = y.astype(int)
```

```
Loading MNIST dataset...
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Full dataset: (70000, 784)
```

```
X, y = X[:10000], y[:10000]
X_train,X_test,y_train,y_test = train_test_split(
X, y, test_size=0.2, random_state=42, stratify=y)
print(f'Train: {X_train.shape} Test: {X_test.shape}')
```

```
Train: (8000, 784) Test: (2000, 784)
```

```
scaler = StandardScaler()
Xtr_s = scaler.fit_transform(X_train.astype(float))
Xte_s = scaler.transform(X_test.astype(float))
```

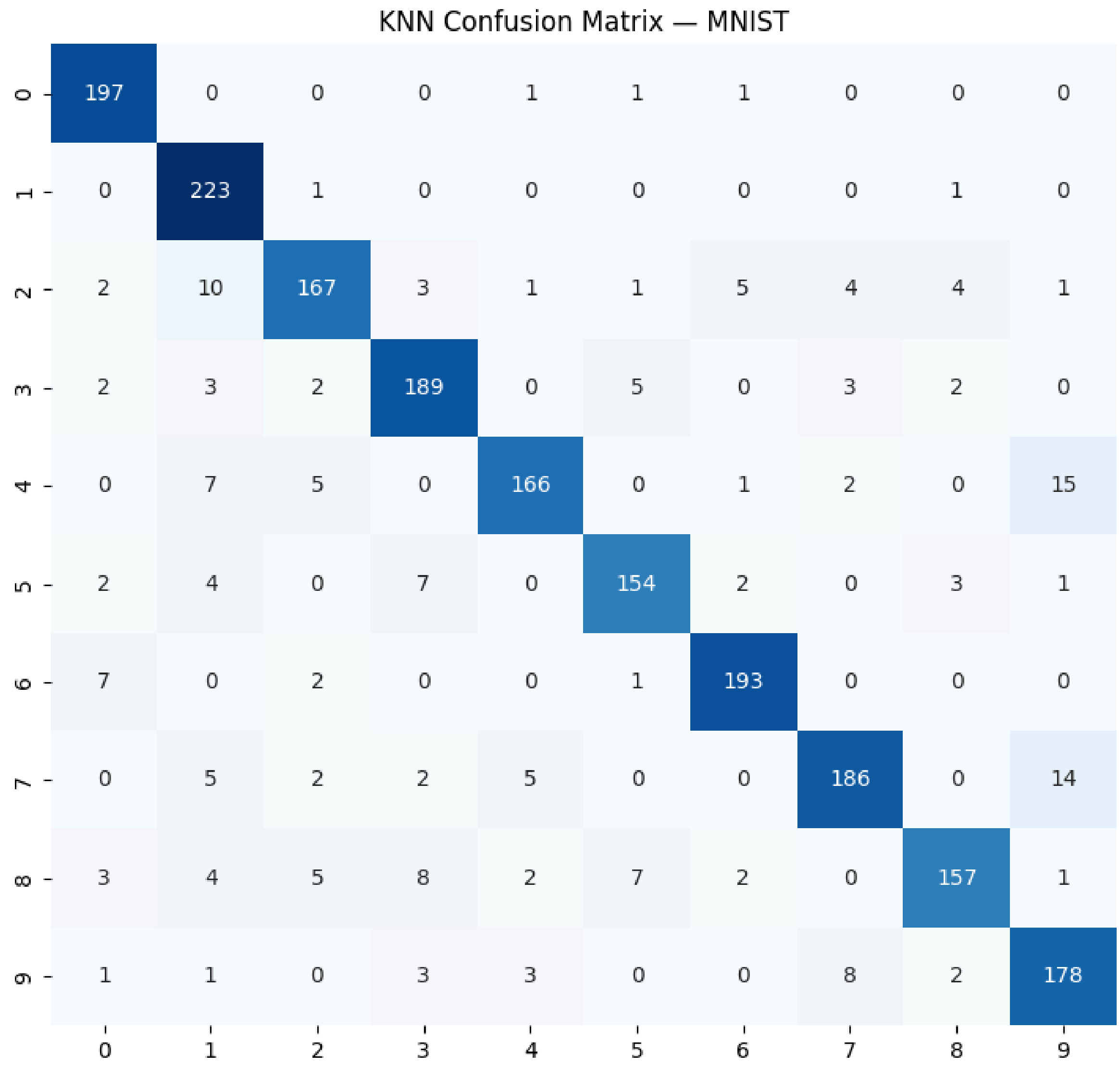
```
knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean',
algorithm='ball_tree', n_jobs=-1)
knn.fit(Xtr_s, y_train)
```

```
KNeighborsClassifier
KNeighborsClassifier(algorithm='ball_tree', metric='euclidean', n_jobs=-1)
```

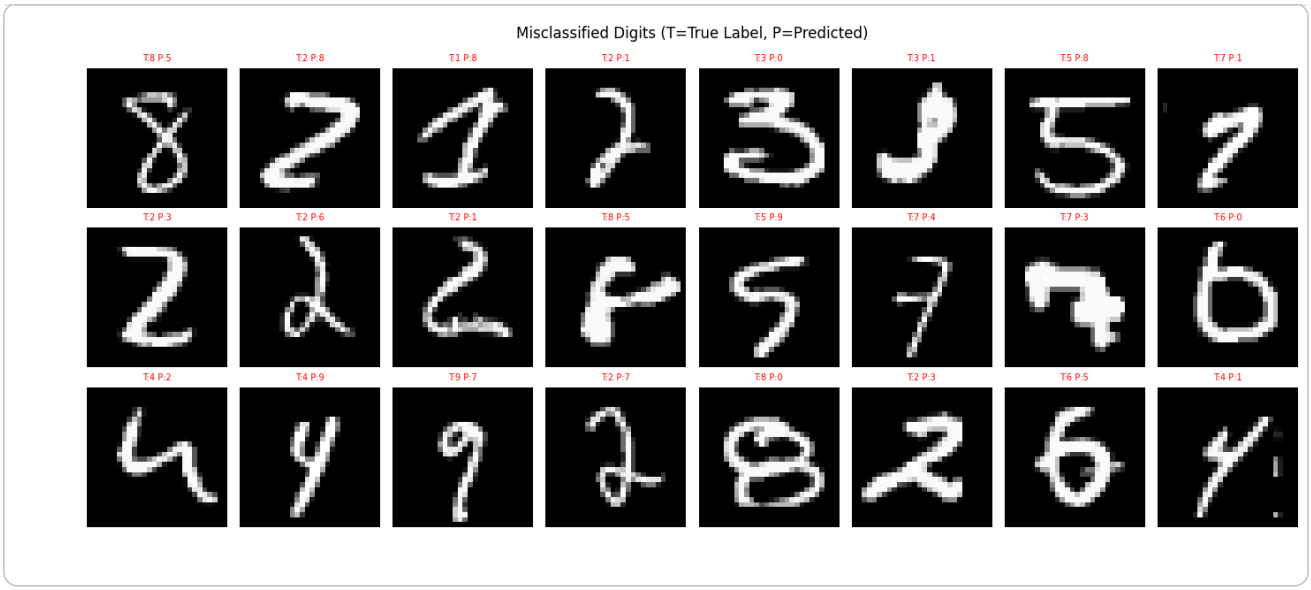
```
y_pred = knn.predict(Xte_s)
print(f'Test Accuracy: {accuracy_score(y_test, y_pred)*100:.2f}%')
print(classification_report(y_test, y_pred))
```

```
Test Accuracy: 90.50%
              precision    recall  f1-score   support

           0       0.92      0.98      0.95       200
           1       0.87      0.99      0.93       225
           2       0.91      0.84      0.87       198
           3       0.89      0.92      0.90       206
           4       0.93      0.85      0.89       196
           5       0.91      0.89      0.90       173
           6       0.95      0.95      0.95       203
           7       0.92      0.87      0.89       214
           8       0.93      0.83      0.88       189
           9       0.85      0.91      0.88       196

    accuracy                           0.91      2000
   macro avg       0.91      0.90      0.90      2000
weighted avg       0.91      0.91      0.90      2000
```

```
# ■■ Confusion matrix ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(9,7))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
xticklabels=range(10), yticklabels=range(10))
plt.title('KNN Confusion Matrix — MNIST'); plt.tight_layout(); plt.show()
```




```
wrong = np.where(y_pred != y_test)[0]
fig, axes = plt.subplots(3,8,figsize=(14,6))
for i,ax in enumerate(axes.flat):
  if i < len(wrong):
    ax.imshow(X_test[wrong[i]].reshape(28,28), cmap='gray')
    ax.set_title(f'T:{y_test[wrong[i]]} P:{y_pred[wrong[i]]}',
    color='red', fontsize=7)
  ax.axis('off')
plt.suptitle('Misclassified Digits (T=True Label, P=Predicted)')
plt.tight_layout(); plt.show()
```



Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.